

RAG Pipeline — Design Decisions

Corpus: Law of the Republic of Armenia on Electronic Communications
Source: [ARLIS act 1869](#) · local `data/law.pdf` (provenance) + `data/law.md` (canonical parse input)

This document explains **why** each stage is built the way it is, and **where** the principal mechanisms live in the code. The same retrieve → assemble → generate path powers both the Ask UI and the Benchmark tab, so model comparison is fair.

1. Document ingestion

Decision: Ingest a **local UTF-8 extract** (`data/law.md`), not a live ARLIS scrape at query or ingest time.

Why

Alternative	Why we rejected it
Live ARLIS HTML scrape	Fragile DOM (nav chrome, incorporation panels, bilingual toggles); non-reproducible across reviewers
PDF-only at query time	Harder to parse deterministically; slow and noisy for article boundaries
Local markdown extract	Stable offline corpus, easy article/chapter parsing, PDF kept as provenance

Flow

- Optional: `scripts/extract-pdf.ts` (`npm run extract-pdf`) dumps PDF text via `unpdf` into `data/law.md`.
- `src/lib/ingest/load-law.ts` loads `law.md` and strips ARLIS chrome (nav header + metadata footer).
- `src/lib/ingest/parse-articles.ts` splits on `| **Հոդված N.** | **Title** |` table headers and tracks `Գ Լ ՈՒ Խ` chapter labels.
- Sanity checks: ingest fails if Article 22 is missing or fewer than ~50 articles parse (bad extract).

Embeddings & storage (at ingest)

- Model: OpenAI `text-embedding-3-large` at **1536** dimensions → Neon **pgvector** (HNSW, cosine).
- Chosen for strong multilingual / cross-lingual behavior (EN questions over an HY-only corpus) without hosting a GPU embedding service.
- 1536-d is a quality/size sweet spot vs full 3072 on Neon.
- Parent embeddings use a ~2500-char body preview (`embeddingSourceText` in `src/lib/rag/embed.ts`) so long articles stay under the embedding token budget; children carry `Հոդված N. Title` + point text. Every embedding input is also hard-capped at 3,800 chars (`MAX_EMBED_CHARS`) because Armenian text is ~2 tokens/char on OpenAI's tokenizer.
- Batches of 16 with a 350 ms pause; each call uses exponential backoff on 429/timeout (`src/lib/util/backoff.ts`).
- Post-ingest verification: a dominance probe must retrieve Article 22 in the top 5, otherwise ingest fails.

Implemented in: `scripts/ingest.ts`, `src/lib/ingest/*`, `src/lib/rag/embed.ts`, Prisma `LawChunk`, HNSW index in `prisma/migrations/20250920120000_init/migration.sql`.

2. Chunking strategy

Decision: **Article parent + paragraph/point children**. Citations always resolve to the parent **article number**.

Why

- The graded citation metric is “correct **article**,” not paragraph id — chunk identity must match what users and the benchmark expect.
- Armenian statutory structure (`1` , `ա`), long definition articles) is lossy under naive fixed-size windows.
- Long articles (e.g. Art. 2 definitions, Art. 5 regulator powers) need precise child passages for retrieval without losing citation identity.

Mechanics (`src/lib/ingest/chunk.ts`)

- One **parent** chunk per article: title + full body (citation identity + fallback).
- Child** chunks split on numbered/lettered points; soft-capped ~800–1200 characters (`MAX_CHILD_CHARS = 1200`).
- Parents and children are both embedded; retrieval later diversifies by article so one long article cannot monopolize context.

We deliberately **do not** use fixed-size-only chunking as the primary strategy.

3. Retrieval

Decision: Hybrid retrieve → lexical rerank → article diversification → confidence gate. For English questions, **always** translate EN→HY with a statutory glossary and re-search.

Pipeline (`src/lib/rag/retrieve.ts`)

1. Detect language (`src/lib/rag/language.ts`).
2. Embed the query; fetch **top-32** cosine neighbors (`TOP_K_FETCH`).
3. Merge with **keyword / stem** ILIKE candidates (≤ 8 per stem, so a broad stem like սահլազն cannot crowd out a rare one like զեղշ) and **article-title** ILIKE candidates. English hints (tariff, interconnect, dominant, ...) map to Armenian stems before translation.
4. **Lexical rerank** (stem hits, topic-stem title hits, near-exact title match).
5. Keep **5** chunks (`TOP_K_KEEP`) with **article diversity** (≤ 4 articles, ≤ 2 chunks/article). Articles whose title clearly matches the query are promoted as parents; otherwise point-level children are preferred over their parent. Selected children are then **replaced by the full parent article** so the model sees complete article text.
6. If the question is English: translate EN→HY (`translateQueryToArmenian`), re-search, prefer HY results when English distance is weak (> 0.55) or HY is within $+0.02$ of English.
7. If best post-rerank distance > 0.62 (`LOW_CONFIDENCE_DISTANCE`), clear context → generator must refuse.

Why

Choice	Rationale
Fetch 32 then diversify	Rare legal stems often sit just outside a tiny top-k
Keyword + lexical boost	Sparse statutory terms where pure vectors fail on Armenian morphology
Article diversity	Avoid flooding the prompt with one article’s children
Always EN→HY translate	Cheap safety net for an Armenian-only corpus without re-indexing an English translation of the whole law
Distance confidence gate	Reduces adversarial “answer from weak neighbors”; Ask also filters citations to retrieved article numbers (<code>src/lib/rag/citations.ts</code>)

Implemented in: `src/lib/rag/retrieve.ts` , `src/lib/rag/language.ts` , citation filter in `src/lib/rag/citations.ts` + Ask/Benchmark routes.

4. Context assembly

Decision: Tag every retrieved block with an explicit article header before it enters the prompt.

```
[Հոդված {n} – {title}]
{content}
```

Blocks are joined with `---` separators. Ask and Benchmark assemble the **full** context; truncation happens later, per provider, in `generateAnswer` (`assembleContext` also accepts an optional `maxChars` , currently unused by the routes).

Why

- Makes article identity unambiguous so models cite the same numbers the UI extracts (`Հոդված N / Article N`).
- Truncation is provider-aware at generation time: only Groq gets an 8,000-char context budget (`truncateContext` keeps earlier, better-ranked blocks whole and soft-cuts the last one) so free-tier TPM (~8k) does not reject the prompt. Other providers get the full context.

Implemented in: `src/lib/rag/assemble.ts` (`assembleContext` , `uniqueArticles`), `truncateContext` in `src/lib/rag/generate.ts` .

5. Answer generation

Decision: One **shared grounded system prompt** for all providers; answer in the user’s language; refuse when context is empty, low-confidence, or out of scope. Underspecified “this article” questions get a **deterministic clarification** (no LLM paraphrase of weak neighbors).

Underspecified questions (`src/lib/rag/underspecified.ts`): a deictic reference («այս / սկսած / սույն հոդված», “this / that article”) without an article number is detected **before retrieval**. Retrieval and the LLM call are skipped, and a fixed HY/EN clarification message is returned with no citations.

Prompt rules (`buildGroundedPrompt` in `src/lib/rag/generate.ts`)

- Ground every substantive claim in retrieved excerpts only.
- Cite as «Հոդված N» / Article N; never invent articles, fines, rates, or obligations.
- Refuse clearly when context is insufficient or the topic is outside this law.
- Never cite an article number that does not appear in the retrieved excerpts.

Providers (task requirement: ≥ 3 free-tier APIs on the same RAG path)

Provider	Model	Role in the product
OpenAI	<code>gpt-4o-mini</code>	Default Ask model; strong instruction following
Google Gemini	<code>gemini-3.6-flash</code>	Multilingual / free-tier contrast
Groq	<code>qwen/qwen3.8-27b</code>	Groq Cloud contrast (<code>max_tokens=768</code> + 8,000-char context for free-tier TPM; override model with <code>GROQ_MODEL</code>)
xAI Grok	<code>grok-4.3</code>	Optional fourth column (separate from Groq); not in the latest benchmark because the configured key is invalid

Each generator returns { `text`, `promptTokens`, `completionTokens`, `ttftMs`, `totalMs`, `status` } so the Benchmark tab can collect answer/citation accuracy, hallucination rate, latency, tokens, paid-rate cost, and failure rate. Cost uses public list prices in `src/lib/llm/pricing.ts` even on free tier.

Retry policy: `generateAnswer` retries `rate_limit` / `timeout` via `withBackoff` (exponential + jitter). Exhausted retries surface as structured failure status — not silent drops.

Citations after generation: `extractCitations` pulls Հոդված N / Article N from the answer, and both Ask and Benchmark drop any number not in the retrieved set. Ask then loads full parent text (`loadArticleBodies` in `src/lib/rag/articles.ts`) for the cited-article cards and the “Related articles” list.

Implemented in: `src/lib/rag/generate.ts`, `src/lib/llm/{openai,gemini,groq,grok,index}.ts`, Ask API `src/app/api/ask/route.ts`, Benchmark API `src/app/api/benchmark/route.ts`.

6. How the pipeline performs (latest benchmark)

Run `cmuf7bi4d0000jdyrtcdc8yc4` (2026-09-24, 19 gold items $\times$ 3 providers, `gpt-4o-mini` judge); full numbers in [EVALUATION REPORT.md](#).

- **Retrieval:** Recall@k on answerable items is **92.3%** (12/13). English answerable is 7/7, so the EN→HY translation path works; the only miss is hy-02 (Art. 17 not retrieved). hy-06 finds Art. 2 but ranks it last behind Arts 59 / 31 / 53.
- **Refusals:** every provider refused or asked for clarification on all 6 adversarial items. The distance confidence gate did not fire on any of them (only adv-06 had empty context, via the underspecified check), so the out-of-scope and near-miss refusals came from the grounded prompt.
- **Known gap:** the citation filter only restricts citations to the *retrieved* set. Gemini and Groq still cited retrieved articles inside refusals on 4 of 6 adversarial items; OpenAI did not. Dropping citations when the answer is a refusal would close this.

End-to-end path

Question

- `detectLanguage`
- underspecified check —(deictic, no number)—→ fixed clarification, no retrieval
- `retrieve` (vector + keyword + title + EN→HY + confidence gate)
- `assembleContext` (tagged articles)
- `generateAnswer` (shared prompt → provider-specific context budget → chosen provider + backoff)
- `extractCitations` → filter to retrieved set
- Ask UI (`loadArticleBodies` for cards) | Benchmark scorer (`src/lib/eval`, `eval/questions.json`)

Principal code map

data/law.md	← canonical corpus
scripts/ingest.ts	← parse → chunk → embed → Neon
src/lib/ingest/*	← load, parse articles, parent/child chunks
src/lib/rag/embed.ts	← text-embedding-3-large (1536)
src/lib/rag/retrieve.ts	← pgvector + keyword + EN-HY + confidence gate
src/lib/rag/assemble.ts	← tagged context
src/lib/rag/generate.ts	← shared prompt + provider dispatch + retries
src/lib/rag/citations.ts	← extract + filter to retrieved articles
src/lib/rag/underspecified.ts	← "this article" detection + clarification text
src/lib/rag/articles.ts	← full article bodies for citation cards
src/lib/util/backoff.ts	← shared retries
src/app/api/ask/route.ts	← Ask UI backend
src/app/api/benchmark/route.ts	← same pipeline × gold set × providers
src/lib/eval/*	← heuristic + LLM-judge scoring, metric aggregation
eval/questions.json	← 19 claim-based gold items (6 HY, 7 EN, 6 adversarial)